Python Fundamentals

200 Multiple-Choice Questions — Answers and Solutions

This answer guide follows the order and answer choices in the supplied CSV. Each item includes the question, all choices for reference, the correct answer, and a concise explanation.

Source: python_fundamentals_200_mcqs.csv

Operators

Q1. What is the result of 10 / 4?

A. 2
B. 2.5
C. 3
D. 4

Answer: 2.5

Solution: The / operator performs true division: 10 / 4 = 2.5.

Q2. What is the result of 10 // 4?

A. 2
B. 2.5
C. 3
D. 4

Answer: 2

Solution: // performs floor division. 10 / 4 is 2.5, and flooring gives 2.

Q3. What is the result of -7 // 2?

A. -3
B. -4
C. 3
D. 4

Answer: -4

Solution: Flooring moves toward negative infinity, so -7 / 2 = -3.5 becomes -4.

Q4. What is the result of 10 % 3?

A. 0
B. 1
C. 2
D. 3

Answer: 1

Solution: % returns the remainder after division. 10 = 3 × 3 + 1.

Q5. What is the result of 2 ** 3?

A. 5
B. 6
C. 8
D. 9

Answer: 8

Solution: ** is exponentiation, so 2 ** 3 means 2 × 2 × 2 = 8.

Q6. What is the result of -3 ** 2?

A. 9
B. -9
C. -6
D. 6

Answer: -9

Solution: Exponentiation binds more tightly than the unary minus: -3 ** 2 is -(3 ** 2) = -9.

Q7. What does the unary operator not always return?

A. An integer
B. A string
C. A Boolean
D. None

Answer: A Boolean

Solution: not performs a truth-value test and always produces a Boolean, True or False.

Q8. What is the result of Py + thon?

A. Py thon
B. Python
C. Py+thon
D. TypeError

Answer: Python

Solution: The + operator concatenates strings, so 'Py' + 'thon' produces 'Python'.

Q9. What is the result of ha * 3?

A. ha3
B. hahaha
C. ha ha ha
D. TypeError

Answer: hahaha

Solution: Multiplying a string repeats it: 'ha' * 3 produces 'hahaha'.

Q10. What is the result of 2 + 3 * 4?

A. 20
B. 14
C. 24
D. 9

Answer: 14

Solution: Multiplication has precedence over addition, so 2 + (3 × 4) = 14.

Q11. What is the result of (2 + 3) * 4?

A. 14
B. 20
C. 24
D. 9

Answer: 20

Solution: Parentheses are evaluated first: (2 + 3) × 4 = 20.

Q12. What type of value do comparison operators return?

A. Boolean
B. Integer
C. String
D. Tuple

Answer: Boolean

Solution: Comparison expressions evaluate to Boolean values: True or False.

Q13. What is the result of 18 <= 25 < 60?

A. False
B. True
C. 25
D. TypeError

Answer: True

Solution: Chained comparisons mean 18 <= 25 and 25 < 60; both are true.

Q14. Which operator compares object identity?

A. ==
B. is
C. in
D. >=

Answer: is

Solution: is tests whether two references point to the same object (identity).

Q15. Which operator compares value equality?

A. is
B. ==
C. not
D. in

Answer: ==

Solution: == compares the values of two objects for equality.

Q16. What does and return when its first operand is falsy?

A. Always True
B. The first falsy operand
C. The last truthy operand
D. None

Answer: The first falsy operand

Solution: and stops at the first falsy operand and returns that operand itself.

Q17. What is the result of an empty string or fallback?

A. An empty string
B. fallback
C. False
D. None

Answer: fallback

Solution: An empty string is falsy, so the or expression returns its fallback operand.

Q18. What does name in profile test when profile is a dictionary?

A. Values
B. Keys
C. Both keys and values
D. Object identity

Answer: Keys

Solution: Membership in a dictionary tests its keys; values require an explicit values() check.

Q19. What does the walrus operator do?

A. Compares two values
B. Assigns inside an expression
C. Performs floor division
D. Creates a generator

Answer: Assigns inside an expression

Solution: The walrus operator := assigns a value while also returning that value in an expression.

Q20. What is the result of 5 & 3?

A. 0
B. 1
C. 6
D. 7

Answer: 1

Solution: Bitwise AND compares bits: 5 (101) & 3 (011) = 1 (001).

Functions

Q21. What does def create?

A. A function object
B. A loop
C. A module
D. A type conversion

Answer: A function object

Solution: A def statement creates a function object and binds its name to it.

Q22. When does a function body normally execute?

A. When def is parsed
B. When the function is called
C. When the file is saved
D. When an argument is defined

Answer: When the function is called

Solution: def creates the function; its body runs later when the function is called.

Q23. What is a parameter?

A. A value in a call
B. A name in a function definition
C. A returned tuple
D. A global variable

Answer: A name in a function definition

Solution: A parameter is the name in the function definition that receives an argument.

Q24. What does return do?

A. Repeats the function
B. Ends the function and sends a value back
C. Creates a parameter
D. Changes global scope

Answer: Ends the function and sends a value back

Solution: return immediately ends the current function and sends a value to its caller.

Q25. What does a function with no return statement return?

A. 0
B. False
C. None
D. An empty tuple

Answer: None

Solution: If execution reaches the end without return, Python returns None implicitly.

Q26. What type receives comma-separated multiple return values?

A. List
B. Tuple
C. Set
D. Dictionary

Answer: Tuple

Solution: Multiple comma-separated return values are packed into a tuple.

Q27. What do positional arguments use?

A. Parameter order
B. Parameter type names
C. Only keyword names
D. A dictionary

Answer: Parameter order

Solution: Positional arguments are matched to parameters from left to right.

Q28. What do keyword arguments use?

A. Parameter names
B. List indexes
C. Object identities
D. Only positional order

Answer: Parameter names

Solution: Keyword arguments identify their target parameters by name.

Q29. What happens when a caller omits an argument with a default?

A. A default value is used
B. The function is skipped
C. None is always returned
D. The argument becomes global

Answer: A default value is used

Solution: If the caller omits a parameter with a default, that default value is used.

Q30. Where must required parameters appear relative to default parameters?

A. After them
B. Before them
C. Only inside args
D. They cannot appear together

Answer: Before them

Solution: Required parameters must precede default parameters in a function signature.

Q31. When are default argument objects evaluated?

A. On every call
B. When the def statement runs
C. After the function returns
D. Only when the argument is passed

Answer: When the def statement runs

Solution: Default expressions are evaluated once, when the def statement executes.

Q32. What is the mutable default argument trap?

A. A mutable default is reused across calls
B. Mutable values cannot be passed
C. Defaults become immutable
D. The function gets no arguments

Answer: A mutable default is reused across calls

Solution: A mutable default is one object reused by later calls, so changes can accumulate.

Q33. Which pattern gives a fresh list for each call?

A. items=[]
B. items=None followed by creating a list inside
C. items=()
D. items=global_list

Answer: items=None followed by creating a list inside

Solution: Using None as the default and creating a list inside the function gives each call a fresh list.

Q34. What does slash mean in a function signature?

A. Parameters before it are positional-only
B. Parameters after it are positional-only
C. All parameters are optional
D. The function returns a tuple

Answer: Parameters before it are positional-only

Solution: In a signature, / marks all parameters before it as positional-only.

Q35. What does star mean before a keyword-only parameter section?

A. Following parameters must be passed by keyword
B. Following parameters are positional-only
C. The function becomes recursive
D. Arguments become a list

Answer: Following parameters must be passed by keyword

Solution: A bare * ends positional parameters; parameters after it must be supplied by keyword.

Q36. What type does star args collect extra positional arguments into?

A. List
B. Tuple
C. Dictionary
D. Set

Answer: Tuple

Solution: Extra positional arguments are collected into the tuple named by *args.

Q37. What type does double-star kwargs collect extra keyword arguments into?

A. Tuple
B. Dictionary
C. List
D. String

Answer: Dictionary

Solution: Extra keyword arguments are collected into the dictionary named by **kwargs.

Q38. What does star values do in a function call?

A. Unpacks positional values
B. Unpacks keyword names
C. Creates a default
D. Suppresses errors

Answer: Unpacks positional values

Solution: A single * in a call unpacks an iterable into positional arguments.

Q39. What does double-star details do in a function call?

A. Unpacks keyword values
B. Unpacks list indexes
C. Copies nested objects
D. Creates a closure

Answer: Unpacks keyword values

Solution: A ** in a call unpacks a mapping into keyword arguments.

Q40. What happens when a function mutates a shared list argument?

A. The caller can see the mutation
B. The caller always sees a copy
C. The function becomes global
D. The list becomes a tuple

Answer: The caller can see the mutation

Solution: The parameter refers to the same mutable list, so in-place mutation is visible to the caller.

Q41. What happens when a function rebinds its parameter to a new object?

A. The caller's name is rebound
B. Only the local parameter is rebound
C. All aliases are deleted
D. The old object changes

Answer: Only the local parameter is rebound

Solution: Rebinding changes only the local parameter name; it does not rebind the caller's name.

Q42. What is the LEGB lookup order?

A. Local, Enclosing, Global, Built-in
B. Global, Local, Enclosing, Built-in
C. Local, Global, Built-in, Enclosing
D. Built-in, Global, Enclosing, Local

Answer: Local, Enclosing, Global, Built-in

Solution: Python resolves names in Local, Enclosing, Global, then Built-in scopes: LEGB.

Q43. What does global count allow inside a function?

A. Assignment to the module-level count
B. Creation of a new local list
C. Calling count as a function
D. Conversion of count to a string

Answer: Assignment to the module-level count

Solution: global declares that assignments to count target the module-level name.

Q44. What does nonlocal count target?

A. The nearest enclosing function's name
B. The module namespace
C. The built-in namespace
D. A new local name

Answer: The nearest enclosing function's name

Solution: nonlocal targets the nearest binding in an enclosing function, not the module scope.

Q45. What error can occur when a local name is read before its local assignment?

A. NameError
B. UnboundLocalError
C. StopIteration
D. RecursionError

Answer: UnboundLocalError

Solution: An assignment makes the name local throughout the function; reading it first raises UnboundLocalError.

Variables

Q46. In Python, a variable name primarily acts as what?

A. A label bound to an object
B. A fixed raw memory box
C. A CPU register
D. A permanent type

Answer: A label bound to an object

Solution: A variable name is a label or reference bound to an object, not a fixed storage box.

Q47. What does dynamic typing allow a name to do?

A. Be rebound to objects of different types
B. Change the object's identity without assignment
C. Become a keyword
D. Avoid all runtime types

Answer: Be rebound to objects of different types

Solution: Dynamic typing lets the same name be rebound to objects of different types at runtime.

Q48. Which characters may appear in an identifier?

A. Letters, digits, and underscores
B. Spaces and hyphens
C. Only digits
D. Only uppercase letters

Answer: Letters, digits, and underscores

Solution: Identifiers may contain letters, digits, and underscores, subject to starting-character rules.

Q49. Which identifier is invalid because it starts with a digit?

A. 2nd_place
B. place_2
C. _place
D. place

Answer: 2nd_place

Solution: Identifiers cannot start with a digit, so 2nd_place is invalid syntax.

Q50. Which statement about Python keywords is correct?

A. An identifier cannot be a Python keyword
B. Keywords can contain spaces
C. Keywords are ignored in identifiers
D. Every identifier must be a keyword

Answer: An identifier cannot be a Python keyword

Solution: Reserved Python keywords have syntactic meaning and cannot be used as identifiers.

Q51. What does case sensitivity mean for identifiers?

A. name and Name are different
B. name and Name are always aliases
C. Capitalization is ignored
D. Only class names are case-sensitive

Answer: name and Name are different

Solution: Python is case-sensitive, so name and Name refer to potentially different names.

Q52. What naming convention is recommended for variables?

A. snake_case
B. PascalCase
C. kebab-case
D. UPPERCASEWITHNODIVIDERS

Answer: snake_case

Solution: PEP 8 recommends lowercase words separated by underscores for ordinary variables.

Q53. What does assigning second = first for a list create?

A. Another reference to the same object
B. A deep copy
C. An immutable tuple
D. A new function

Answer: Another reference to the same object

Solution: Assignment copies the reference, so both names point to the same list object.

Q54. What is rebinding?

A. Making a name refer to another object
B. Changing every alias at once
C. Deleting an object from memory
D. Changing a keyword

Answer: Making a name refer to another object

Solution: Rebinding makes a name refer to a different object; it does not mutate the old object.

Q55. What does mutation change?

A. An existing object's state
B. Only a variable's spelling
C. The Python parser
D. The function signature

Answer: An existing object's state

Solution: Mutation changes the state of an existing object, such as adding an item to a list.

Q56. What usually happens when an immutable integer is incremented?

A. A new integer is bound
B. The integer is changed in place
C. All integer aliases change
D. The name is deleted

Answer: A new integer is bound

Solution: Integers are immutable, so incrementing binds the name to a new integer object.

Q57. What does multiple assignment bind?

A. Several names in one statement
B. Only global names
C. Only function parameters
D. A name to a keyword

Answer: Several names in one statement

Solution: Multiple assignment binds several names in one statement, often by unpacking values.

Q58. What does chained assignment bind?

A. Each name to the same object
B. Each name to a separate deep copy
C. Only the last name
D. No names

Answer: Each name to the same object

Solution: Chained assignment binds each name to the same right-hand object reference.

Q59. What type does a starred unpacking target receive?

A. List
B. Tuple
C. Set
D. Dictionary

Answer: List

Solution: The starred unpacking target gathers remaining items into a new list.

Q60. What does a variable annotation do at runtime?

A. Documents an expected type but does not enforce it
B. Always converts the value
C. Makes the name constant
D. Copies the object

Answer: Documents an expected type but does not enforce it

Solution: An annotation records an expected type for tools and readers; ordinary Python does not enforce it.

Datatypes

Q61. What does every Python value have?

A. A type and identity
B. Only a memory address
C. Only a variable name
D. A required annotation

Answer: A type and identity

Solution: Every Python object has a type and an identity; its state is represented by its value or contents.

Q62. Which type represents whole numbers?

A. int
B. float
C. complex
D. str

Answer: int

Solution: int represents whole-number values, such as 0, 7, or -12.

Q63. Which type represents binary floating-point values?

A. float
B. int
C. bool
D. bytes

Answer: float

Solution: float represents binary floating-point values, such as 3.14.

Q64. Which suffix represents the imaginary part of a complex number?

A. i
B. j
C. c
D. z

Answer: j

Solution: Python uses j (or J) to mark the imaginary component of a complex literal.

Q65. What is true about bool in Python?

A. It is a subclass of int
B. It is a subclass of str
C. It has three values
D. It cannot be added

Answer: It is a subclass of int

Solution: bool is implemented as a subclass of int, with True behaving numerically like 1 and False like 0.

Q66. What does None represent?

A. No value or no result
B. The integer zero
C. An empty string only
D. A false object with no type

Answer: No value or no result

Solution: None is Python's singleton meaning absence of a value or result; it is not the same as zero.

Q67. Which is the correct way to check for None?

A. value == None
B. value is None
C. value in None
D. type(value) == False

Answer: value is None

Solution: is None tests identity with the singleton None and is the recommended check.

Q68. What kind of sequence is a string?

A. Mutable Unicode sequence
B. Immutable Unicode sequence
C. Unordered numeric set
D. Mutable byte array

Answer: Immutable Unicode sequence

Solution: A string is an immutable Unicode sequence, so its characters cannot be changed in place.

Q69. What is a list?

A. An ordered mutable sequence
B. An immutable mapping
C. An unordered immutable sequence
D. A binary-only type

Answer: An ordered mutable sequence

Solution: A list preserves order and supports in-place changes, making it an ordered mutable sequence.

Q70. What is a tuple?

A. An ordered immutable sequence
B. A mutable set
C. A key-value mapping
D. A byte array

Answer: An ordered immutable sequence

Solution: A tuple preserves order but cannot be changed after creation, so it is immutable.

Q71. How is a one-item tuple written?

A. (10)
B. (10,)
C. [10]
D. {10}

Answer: (10,)

Solution: The comma creates the tuple; parentheses alone do not. Thus (10,) has one item.

Q72. What does range(2, 8, 2) exclude?

A. 2
B. 4
C. 6
D. 8

Answer: 8

Solution: range stops before its stop argument, so range(2, 8, 2) yields 2, 4, and 6.

Q73. What does a dictionary map?

A. Unique hashable keys to values
B. Values to only integers
C. Indexes to characters
D. Mutable keys to names

Answer: Unique hashable keys to values

Solution: A dictionary maps unique hashable keys to values.

Q74. What order does a dictionary preserve?

A. Insertion order
B. Sorted order only
C. Reverse hash order
D. No observable order

Answer: Insertion order

Solution: Modern Python dictionaries preserve insertion order when iterated.

Q75. What does dictionary membership test?

A. Keys
B. Values
C. Both equally
D. Object identities

Answer: Keys

Solution: The in operator on a dictionary checks keys by default.

Q76. What property defines a set's elements?

A. They are unique hashable values
B. They are always ordered
C. They are always mutable lists
D. They must be strings

Answer: They are unique hashable values

Solution: A set stores unique hashable values and automatically removes duplicate insertions.

Q77. What is a frozenset?

A. An immutable set
B. A sorted list
C. A mutable dictionary
D. A byte sequence

Answer: An immutable set

Solution: A frozenset has set semantics but is immutable, so it can itself be hashed when its elements are hashable.

Q78. What is true about bytes?

A. It is an immutable sequence of integers from 0 to 255
B. It is a mutable Unicode string
C. It stores only floating values
D. It is an unordered set

Answer: It is an immutable sequence of integers from 0 to 255

Solution: bytes is immutable and represents integer byte values from 0 through 255.

Q79. What is the mutable counterpart of bytes?

A. bytearray
B. memoryview
C. frozenset
D. range

Answer: bytearray

Solution: bytearray has the same byte-value range as bytes but supports in-place modification.

Q80. Which type is usually mutable?

A. list
B. tuple
C. str
D. bytes

Answer: list

Solution: Lists are designed for in-place operations such as append, item assignment, and removal.

Control flow

Q81. Which value is falsy?

A. [10]
B. 1
C. Python
D. []

Answer: []

Solution: An empty list is falsy in a Boolean context; non-empty lists are truthy.

Q82. How does Python select among if and elif branches?

A. It runs every true branch
B. It runs the first true branch
C. It runs only else
D. It chooses randomly

Answer: It runs the first true branch

Solution: Python tests if/elif conditions in order and executes only the first branch whose condition is true.

Q83. What does a conditional expression choose?

A. One of two values
B. A function frame
C. A dictionary key
D. A generator state

Answer: One of two values

Solution: A conditional expression has the form x if condition else y and selects exactly one of two values.

Q84. How does Python evaluate 18 <= 25 < 60?

A. False
B. True
C. 25
D. TypeError

Answer: True

Solution: Chained comparison is equivalent to 18 <= 25 and 25 < 60, so it is True.

Q85. What is the precedence order among not, and, and or?

A. or, and, not
B. not, and, or
C. and, or, not
D. They have no precedence

Answer: not, and, or

Solution: Boolean precedence is not first, then and, then or; parentheses can make intent explicit.

Q86. What does short-circuiting allow and to do?

A. Stop at the first falsy operand
B. Always evaluate every operand
C. Convert every operand to an integer
D. Skip only truthy operands

Answer: Stop at the first falsy operand

Solution: and stops as soon as it finds a falsy operand, avoiding evaluation of later operands.

Q87. What does a for loop consume one item at a time from?

A. An iterable
B. Only a list
C. Only a dictionary
D. A function frame

Answer: An iterable

Solution: for obtains values from an iterable, such as a list, string, range, or generator.

Q88. What does range(start, stop, step) exclude?

A. start
B. stop
C. step
D. All values

Answer: stop

Solution: The stop value in range is exclusive, so it is never produced.

Q89. What does enumerate() provide?

A. An index and an item
B. Only indexes
C. Only values
D. Keys and values from a dictionary

Answer: An index and an item

Solution: enumerate pairs each item with its index, for example (0, first_item).

Q90. When does zip() stop?

A. At the longest iterable
B. At the shortest iterable
C. After one item always
D. Only when a list is empty

Answer: At the shortest iterable

Solution: zip cannot produce a pair after any input ends, so it stops at the shortest iterable.

Q91. What controls a while loop?

A. A condition that remains true
B. A fixed tuple length
C. A dictionary key
D. A function annotation

Answer: A condition that remains true

Solution: A while loop repeats while its condition remains truthy; the loop must eventually make it false to finish.

Q92. What does break exit?

A. The nearest loop
B. All functions
C. The module
D. Only the current expression

Answer: The nearest loop

Solution: break exits the nearest enclosing loop, not all loops or the surrounding function.

Q93. What does continue do?

A. Skips the rest of the current iteration
B. Exits every loop
C. Repeats the function definition
D. Stops the program

Answer: Skips the rest of the current iteration

Solution: continue skips the remaining body of the current iteration and starts the next iteration.

Q94. What does pass do?

A. Nothing; it is a placeholder statement
B. Exits the loop
C. Returns None from every function
D. Skips the module

Answer: Nothing; it is a placeholder statement

Solution: pass is a no-operation placeholder used where a statement is syntactically required.

Q95. When does a loop else block run?

A. Only when the loop finishes without break
B. Whenever continue runs
C. Before the loop starts
D. Only after break

Answer: Only when the loop finishes without break

Solution: A loop's else block runs only when the loop ends normally, not when break exits it.

Q96. Which loop does break affect in nested loops?

A. The nearest loop
B. Every outer loop
C. Only the first loop
D. No loop

Answer: The nearest loop

Solution: In nested loops, break applies to the nearest loop containing that break statement.

Q97. How does match choose among cases?

A. The first matching case
B. The last case always
C. Only the wildcard case
D. A random case

Answer: The first matching case

Solution: match checks cases in source order and selects the first case whose pattern and guard match.

Q98. What is case underscore commonly used for?

A. A fallback pattern
B. A loop counter
C. A type conversion
D. A function return

Answer: A fallback pattern

Solution: The underscore pattern _ is commonly a catch-all fallback in a match statement.

Q99. What does any([]) return?

A. True
B. False
C. None
D. An empty list

Answer: False

Solution: any([]) is False because no element in the empty iterable is truthy.

Q100. What does all([]) return?

A. True
B. False
C. None
D. An empty tuple

Answer: True

Solution: all([]) is True by vacuous truth: there is no element that violates the condition.

Iterators

Q101. What is an iterable?

A. An object that can produce an iterator
B. An object that must be a generator
C. A value that can only be used once
D. A function with yield

Answer: An object that can produce an iterator

Solution: An iterable can provide an iterator, usually through __iter__, so it can be traversed.

Q102. What is an iterator?

A. An object that remembers traversal state and produces the next value
B. Any list
C. Only a tuple
D. A reusable dictionary copy

Answer: An object that remembers traversal state and produces the next value

Solution: An iterator maintains traversal state and supplies its next value when asked.

Q103. What does iter(numbers) usually return when numbers is a list?

A. A list copy
B. A list iterator
C. The integer length
D. None

Answer: A list iterator

Solution: iter(list) returns a list_iterator object that traverses the list.

Q104. What is normally true of iter(iterator) is iterator?

A. It is True
B. It is always False
C. It raises TypeError
D. It returns None

Answer: It is True

Solution: An iterator's __iter__ method normally returns itself, so iter(iterator) is iterator is True.

Q105. What does next(iterator) do?

A. Returns the next item
B. Restarts the iterator
C. Copies all remaining items
D. Returns the iterator type only

Answer: Returns the next item

Solution: next(iterator) requests and returns the next item in the iterator's sequence.

Q106. What does an exhausted iterator raise when advanced?

A. ValueError
B. StopIteration
C. IndexError
D. GeneratorExit

Answer: StopIteration

Solution: Once no items remain, next raises StopIteration to signal normal exhaustion.

Q107. Are normal iterators automatically reusable after exhaustion?

A. Yes, they restart
B. No, they are one-pass
C. Only if they contain lists
D. Only inside for

Answer: No, they are one-pass

Solution: Most iterators are one-pass: after exhaustion they do not restart automatically.

Q108. Which operations does a for loop conceptually use?

A. iter() and repeated next()
B. Only len()
C. Only indexing
D. copy() and sort()

Answer: iter() and repeated next()

Solution: A for loop obtains an iterator with iter() and repeatedly calls next() until StopIteration.

Q109. Which methods define a custom iterator?

A. __iter__() and __next__()
B. __start__() and __stop__()
C. __get__() and __set__()
D. __enter__() and __exit__()

Answer: __iter__() and __next__()

Solution: A custom iterator implements __iter__() and __next__().

Q110. What should a custom iterator return from __iter__()?

A. Itself
B. A new class definition
C. None always
D. A dictionary

Answer: Itself

Solution: For an iterator object, __iter__() should return self so it can be used by for and iter().

Q111. What happens when list(iterator) consumes an iterator?

A. The iterator advances through its values
B. The iterator becomes a reusable list
C. The iterator is copied twice
D. The iterator becomes a tuple

Answer: The iterator advances through its values

Solution: list(iterator) repeatedly consumes next(), advancing the iterator through all remaining values.

Q112. What does a second list(iterator) usually produce after exhaustion?

A. The original values
B. An empty list
C. None
D. StopIteration as a list

Answer: An empty list

Solution: After exhaustion there are no remaining values, so a second list(iterator) is empty.

Q113. How can an iterable be traversed again after its iterator is exhausted?

A. Create a new iterator from the iterable
B. Call next() on the exhausted iterator
C. Use id() on it
D. Convert it to None

Answer: Create a new iterator from the iterable

Solution: Create a fresh iterator from the original reusable iterable, such as iter(numbers).

Q114. What state does an iterator preserve?

A. Its current traversal position
B. The entire source file
C. Only its final value
D. The module's global names

Answer: Its current traversal position

Solution: The iterator stores its current position so the next call continues where the last one stopped.

Q115. When is a generator often simpler than a custom iterator class?

A. When traversal state is simple
B. When no values should be produced
C. When all values must be stored immediately
D. When a dictionary key is required

Answer: When traversal state is simple

Solution: A generator is usually simpler when the traversal state can be expressed with local variables and yield.

Generators and context managers

Q116. What does a function containing yield return when called?

A. A generator iterator
B. A list immediately
C. A tuple immediately
D. None always

Answer: A generator iterator

Solution: Calling a function containing yield creates a generator iterator; its body has not run yet.

Q117. When does a generator function body begin running?

A. When the function is called
B. When next() or iteration begins
C. When the file is imported only
D. When the generator is deleted

Answer: When next() or iteration begins

Solution: Generator execution is lazy and starts on next() or when iteration begins.

Q118. What does yield do to generator execution?

A. Pauses it and preserves local state
B. Ends the entire program
C. Copies the yielded value
D. Converts it to a list

Answer: Pauses it and preserves local state

Solution: yield produces a value and suspends execution while preserving locals and the execution position.

Q119. What happens when next() resumes a generator?

A. It continues until the next yield, return, or exception
B. It always restarts from the first line
C. It returns the complete list
D. It closes the module

Answer: It continues until the next yield, return, or exception

Solution: Resuming with next continues until another yield, a return, or an exception.

Q120. What exception signals normal generator exhaustion?

A. StopIteration
B. ValueError
C. NameError
D. TimeoutError

Answer: StopIteration

Solution: Normal completion of a generator is reported to the iterator protocol as StopIteration.

Q121. Where does a generator return value appear during manual iteration?

A. StopIteration.value
B. The generator __name__
C. The first yielded item
D. The caller globals

Answer: StopIteration.value

Solution: A generator's return expression becomes the value attribute of the final StopIteration.

Q122. What does yield from iterable do?

A. Delegates iteration to another iterable
B. Copies the iterable deeply
C. Stops all generators
D. Converts values to strings

Answer: Delegates iteration to another iterable

Solution: yield from forwards iteration to another iterable and handles its values for the caller.

Q123. What does a generator expression produce?

A. Values lazily
B. A complete list immediately
C. A dictionary key view
D. A class object

Answer: Values lazily

Solution: A generator expression computes values lazily as iteration requests them.

Q124. Which usually stores the full result immediately?

A. A list comprehension
B. A generator expression
C. A generator function before next()
D. An iterator object

Answer: A list comprehension

Solution: A list comprehension builds and stores the complete list immediately.

Q125. What is a lazy pipeline?

A. Stages that produce values only as requested
B. A pipeline that stores every value twice
C. A context manager with no cleanup
D. A loop that never yields

Answer: Stages that produce values only as requested

Solution: A lazy pipeline connects stages that produce each value only when downstream code requests it.

Q126. Which methods form the context manager protocol?

A. __enter__() and __exit__()
B. __iter__() and __next__()
C. __call__() and __repr__()
D. __open__() and __close__()

Answer: __enter__() and __exit__()

Solution: The context-manager protocol uses __enter__() on entry and __exit__() on leaving the with block.

Q127. When does __exit__() run for a successfully entered with block?

A. After the body
B. Before __enter__()
C. Only when an exception occurs
D. Never

Answer: After the body

Solution: After successful entry, __exit__() runs when the with body finishes, including during cleanup.

Q128. What does returning False from __exit__() usually do for an exception?

A. Allows it to propagate
B. Suppresses it
C. Converts it to None
D. Restarts the body

Answer: Allows it to propagate

Solution: Returning False from __exit__ does not suppress an exception, so it propagates normally.

Q129. What does returning True from __exit__() do?

A. Suppresses the handled exception
B. Always re-raises the exception
C. Skips setup
D. Creates a generator

Answer: Suppresses the handled exception

Solution: Returning True from __exit__ tells the with statement that the exception was handled.

Q130. What does contextlib.contextmanager help create?

A. A context manager around a generator function
B. A custom numeric type
C. A dictionary iterator only
D. A recursive function

Answer: A context manager around a generator function

Solution: contextlib.contextmanager turns a generator function with setup/yield/cleanup into a context manager.

Q131. In a contextmanager-decorated function, what usually happens before yield?

A. Setup
B. Final cleanup
C. Exception re-raising only
D. Module import

Answer: Setup

Solution: Code before yield runs on entering the context and is used for setup.

Q132. Where is deterministic cleanup commonly placed in a contextmanager function?

A. A finally block
B. An elif branch
C. A list comprehension
D. A default argument

Answer: A finally block

Solution: A finally block runs whether the body succeeds or raises, making it suitable for deterministic cleanup.

Q133. In with managed(A), managed(B), which manager exits first?

A. A
B. B
C. Both simultaneously
D. Neither

Answer: B

Solution: Nested context managers exit in reverse order: B is entered after A and exits first.

Q134. Which is best for reusable stored values?

A. A list or tuple
B. A one-pass generator only
C. A context manager
D. StopIteration

Answer: A list or tuple

Solution: A list or tuple retains values for reuse, unlike a one-pass iterator that may be exhausted.

Q135. Which tool is intended for deterministic resource cleanup?

A. A context manager
B. A random iterator
C. Garbage collection alone
D. A type annotation

Answer: A context manager

Solution: A context manager deterministically acquires and releases resources around a with block.

Memory Model

Q136. What does a namespace map?

A. Names to objects
B. Types to source files
C. Objects to keywords only
D. Frames to CPUs

Answer: Names to objects

Solution: A namespace maps names to the objects those names currently reference.

Q137. What are the three useful properties of an object?

A. Type, state, and identity
B. Name, line, and keyword
C. Loop, branch, and frame
D. Value, file, and module

Answer: Type, state, and identity

Solution: An object is usefully described by its type, state, and identity.

Q138. What can be true for two separate lists with equal contents?

A. == is True and is is False
B. == is False and is is True
C. Both are always False
D. Both are always True

Answer: == is True and is is False

Solution: Separate lists may contain equal values while being different objects: == is True and is is False.

Q139. What does id(object) represent?

A. An identity value unique during the object's lifetime
B. A permanent application ID
C. The object's type name
D. Its list length

Answer: An identity value unique during the object's lifetime

Solution: id returns an identity value unique among live objects for that object's lifetime.

Q140. What is aliasing?

A. Multiple references point to one mutable object
B. A value is converted to a string
C. A function returns None
D. A list is sorted

Answer: Multiple references point to one mutable object

Solution: Aliasing means multiple references point to the same object, especially important for mutable objects.

Q141. What can mutation through an alias affect?

A. The same object seen through other aliases
B. Only the alias spelling
C. Only immutable values
D. The Python version

Answer: The same object seen through other aliases

Solution: Mutating through one alias changes the shared object, so other aliases observe the change.

Q142. What does rebinding change?

A. One name's reference to an object
B. The old object's state automatically
C. Every object's type
D. The garbage collector algorithm

Answer: One name's reference to an object

Solution: Rebinding changes one name's reference; it does not automatically alter the object previously referenced.

Q143. What does mutation preserve for a mutable object?

A. Its identity
B. Its original value forever
C. Its variable name
D. Its module frame

Answer: Its identity

Solution: Mutation changes contents in place, preserving the mutable object's identity.

Q144. What happens when an immutable string is concatenated?

A. A new string object is produced
B. The original string changes in place
C. All aliases mutate
D. The string becomes a list

Answer: A new string object is produced

Solution: Strings are immutable, so concatenation creates a new string instead of changing the old one.

Q145. When does an object remain reachable?

A. While at least one live reference can reach it
B. Only while its first name exists
C. Only while it is a list
D. Until the next function call

Answer: While at least one live reference can reach it

Solution: An object remains reachable while at least one live reference can still reach it.

Q146. What can garbage collection reclaim?

A. Unreachable objects
B. Every object after each statement
C. Only integers
D. All global names

Answer: Unreachable objects

Solution: Garbage collection can reclaim objects that are no longer reachable from live roots.

Q147. Why can a reference cycle require cyclic garbage collection?

A. Reference counts alone may not remove the cycle
B. Cycles are always syntax errors
C. Lists cannot reference themselves
D. The object is immutable

Answer: Reference counts alone may not remove the cycle

Solution: Reference counting alone can leave a closed cycle with nonzero counts, so cyclic GC is needed.

Q148. Why should garbage-collection timing not manage files or connections?

A. Use deterministic cleanup such as with
B. Garbage collection never runs
C. Files are always immutable
D. Connections cannot be objects

Answer: Use deterministic cleanup such as with

Solution: Resource lifetime should be explicit; use deterministic cleanup such as a with statement rather than GC timing.

Q149. What does assignment usually create?

A. Another reference
B. A recursive copy of all nested objects
C. A new type
D. A new process

Answer: Another reference

Solution: Ordinary assignment creates another reference to the same object; it does not recursively copy it.

Q150. What does a shallow copy create?

A. A new outer object with shared nested references
B. A fully independent object graph
C. Only a new variable name
D. An immutable object always

Answer: A new outer object with shared nested references

Solution: A shallow copy duplicates the outer container but retains references to its nested objects.

Q151. What does a deep copy do to nested mutable objects?

A. Recursively copies them
B. Always aliases them
C. Deletes them
D. Converts them to tuples

Answer: Recursively copies them

Solution: A deep copy recursively copies nested objects, separating the copied object graph.

Q152. What happens with rows = [[0] * 2] * 3 when one row is changed?

A. The repeated inner row references can all change
B. Only the first row can change
C. The rows become tuples
D. The assignment raises TypeError

Answer: The repeated inner row references can all change

Solution: The * 3 repeats the same inner-list reference, so changing one row can change all repeated rows.

Q153. Which construction creates independent rows?

A. [[0] * 2 for _ in range(3)]
B. [[0] * 2] * 3
C. rows = rows
D. rows = None

Answer: [[0] * 2 for _ in range(3)]

Solution: The comprehension creates a new inner list on each iteration, giving rows independent storage.

Q154. What does a function call do with an object argument?

A. Binds a local parameter to the same object
B. Always deep-copies it
C. Always serializes it
D. Deletes the caller's name

Answer: Binds a local parameter to the same object

Solution: Calling a function binds its local parameter to the same object reference; it does not automatically deep-copy it.

Q155. What is the safe rule for interning and identity?

A. Use == for normal value comparison
B. Use is for all equal values
C. Interning applies to every object
D. Identity is a permanent address

Answer: Use == for normal value comparison

Solution: Use == for normal value comparison; identity behavior of interned objects is an implementation detail.

Execution Model

Q156. What is a simplified CPython path from source to execution?

A. Source to tokens/AST to code object/bytecode to frame
B. Source directly to machine memory
C. Source to list to tuple to frame
D. Source to dictionary only

Answer: Source to tokens/AST to code object/bytecode to frame

Solution: A simplified CPython path is source → tokens/AST → code object/bytecode → execution frame.

Q157. What does exec() do?

A. Executes code in a namespace
B. Only parses comments
C. Creates a list iterator
D. Converts text to bytes

Answer: Executes code in a namespace

Solution: exec compiles and executes supplied code in the namespace(s) given to it.

Q158. Why should untrusted input not be passed to exec()?

A. It can run arbitrary code
B. It always returns None
C. It cannot create names
D. It only supports arithmetic

Answer: It can run arbitrary code

Solution: Untrusted text passed to exec can execute arbitrary Python code, so it is unsafe without strong isolation.

Q159. How does Python execute module-level statements?

A. From top to bottom
B. From bottom to top
C. Only inside functions
D. In random order

Answer: From top to bottom

Solution: At module level, Python executes statements sequentially from top to bottom.

Q160. When does a function body run after a def statement?

A. Only when the function is called
B. Immediately before the next statement
C. Only during garbage collection
D. Only when imported twice

Answer: Only when the function is called

Solution: A def statement creates the function; its body runs only when the function is called.

Q161. What does each active function call have conceptually?

A. An execution frame
B. A new operating system process
C. A new Python installation
D. A new source file

Answer: An execution frame

Solution: Each active function call has an execution frame containing its locals and execution state.

Q162. What name-resolution order applies inside nested functions?

A. Local, Enclosing, Global, Built-in
B. Global, Local, Built-in, Enclosing
C. Built-in, Global, Local, Enclosing
D. Enclosing, Built-in, Local, Global

Answer: Local, Enclosing, Global, Built-in

Solution: Nested lookup follows LEGB: Local, Enclosing, Global, then Built-in.

Q163. Why can a function read raise UnboundLocalError even when a global name exists?

A. An assignment in the function makes the name local throughout
B. Globals are never readable
C. The name must be a generator
D. The function has no frame

Answer: An assignment in the function makes the name local throughout

Solution: Any assignment marks the name local throughout that function, so an earlier read raises UnboundLocalError.

Q164. What happens before a function body begins?

A. The callable and argument expressions are evaluated
B. The return value is printed
C. The function is garbage-collected
D. The module is re-imported

Answer: The callable and argument expressions are evaluated

Solution: Python evaluates the callable and argument expressions before entering the function body.

Q165. When are parameters bound?

A. Before the function body runs
B. After the body returns
C. Only after an exception
D. During garbage collection

Answer: Before the function body runs

Solution: Arguments are matched and parameters are bound before the first body statement executes.

Q166. What happens to a prepared return value before a function returns?

A. A finally block runs first
B. It is always discarded
C. The module exits
D. The caller runs first

Answer: A finally block runs first

Solution: A finally clause runs during unwinding, even after a return value has been prepared.

Q167. What happens to an unhandled exception?

A. It unwinds frames until a matching handler is found
B. It becomes a list
C. It restarts the module automatically
D. It is always suppressed

Answer: It unwinds frames until a matching handler is found

Solution: An unhandled exception unwinds call frames until a matching exception handler is found or the program terminates.

Q168. Which methods does a with statement call conceptually?

A. __enter__() and __exit__()
B. __iter__() and __next__()
C. __call__() and __repr__()
D. __open__() and __close__()

Answer: __enter__() and __exit__()

Solution: with invokes __enter__ before the body and __exit__ afterward, including during exception handling.

Q169. How does a for loop obtain successive values?

A. It repeatedly calls next() and handles StopIteration
B. It calls id() only once
C. It deep-copies the iterable
D. It invokes __enter__()

Answer: It repeatedly calls next() and handles StopIteration

Solution: for repeatedly calls next on an iterator and stops when StopIteration is raised.

Q170. What does a generator preserve when suspended?

A. Its execution frame and local state
B. Only its first value
C. The entire operating system
D. No state

Answer: Its execution frame and local state

Solution: A suspended generator keeps its frame, instruction position, and local variables.

Q171. When do decorators normally run?

A. When the decorated function or class is defined
B. Only when the function is called
C. After the process exits
D. Only after garbage collection

Answer: When the decorated function or class is defined

Solution: Decorator expressions and application normally occur when the decorated definition executes.

Q172. What executes during a class statement?

A. The class body executes in a namespace
B. Only method calls execute
C. Nothing executes until an instance exists
D. Only annotations execute

Answer: The class body executes in a namespace

Solution: The class body executes immediately in a temporary namespace while the class statement runs.

Q173. What role does sys.modules have during normal import?

A. It caches imported modules
B. It stores only function arguments
C. It disables all imports
D. It stores bytecode instructions only

Answer: It caches imported modules

Solution: sys.modules caches imported module objects so normal repeated imports can reuse them.

Q174. What does the main-module guard check?

A. Whether __name__ is __main__
B. Whether id() is zero
C. Whether a list is empty
D. Whether a function has yield

Answer: Whether __name__ is __main__

Solution: The main-module guard tests whether __name__ == '__main__'.

Q175. How do normal separate processes differ from threads in memory?

A. Separate processes normally have separate memory spaces
B. Processes always share every object
C. Threads always have separate Python runtimes
D. Processes cannot execute Python

Answer: Separate processes normally have separate memory spaces

Solution: Separate processes normally have separate address spaces; threads within a process share its memory.

Type Conversion

Q176. How are constants normally indicated in Python?

A. UPPER_SNAKE_CASE by convention
B. A constant keyword
C. A leading digit
D. The const statement

Answer: UPPER_SNAKE_CASE by convention

Solution: Python has no enforced const keyword; UPPER_SNAKE_CASE communicates a constant-by-convention name.

Q177. What does Final provide?

A. Static-checker guidance against reassignment
B. Runtime immutability
C. Automatic conversion to a tuple
D. Garbage collection

Answer: Static-checker guidance against reassignment

Solution: Final is primarily for static type checkers; it does not make a value immutable at runtime.

Q178. What does type(value) return?

A. The exact class of the value
B. A Boolean subclass check
C. The object's identity number
D. A string conversion

Answer: The exact class of the value

Solution: type(value) returns the value's exact runtime class.

Q179. What does isinstance(value, Type) accept?

A. The type or one of its subclasses
B. Only exact identity
C. Only strings
D. Only immutable values

Answer: The type or one of its subclasses

Solution: isinstance accepts instances of the requested type and its subclasses.

Q180. What is isinstance(True, int)?

A. True
B. False
C. None
D. TypeError

Answer: True

Solution: bool is a subclass of int, and True is an instance of int, so the result is True.

Q181. What does int("25") return?

A. 25
B. 25.0
C. 25 as a string
D. ValueError

Answer: 25

Solution: The decimal text '25' is parsed as an integer, producing 25.

Q182. What exception can int("2.5") raise?

A. TypeError
B. ValueError
C. NameError
D. StopIteration

Answer: ValueError

Solution: '2.5' is valid float text but not valid integer text, so int raises ValueError.

Q183. What does int("1010", 2) return?

A. 10
B. 1010
C. 12
D. 2

Answer: 10

Solution: With base 2, 1010 represents 1×8 + 0×4 + 1×2 + 0×1 = 10.

Q184. What does int("ff", 16) return?

A. 15
B. 255
C. 16
D. Error

Answer: 255

Solution: With base 16, ff is 15×16 + 15 = 255.

Q185. How does int(3.9) convert the float?

A. Truncates toward zero
B. Rounds to 4
C. Rounds down to 3 only
D. Raises ValueError

Answer: Truncates toward zero

Solution: int converts a float by truncating toward zero; int(3.9) is 3.

Q186. What does float(7) return?

A. 7
B. 7.0
C. 7.0 as a string
D. None

Answer: 7.0

Solution: float converts the integer 7 to the floating-point value 7.0.

Q187. What does str(100) create?

A. The string 100
B. The integer 100
C. The float 100.0
D. A bytes object

Answer: The string 100

Solution: str(100) creates the text '100', not another numeric object.

Q188. What is bool(0)?

A. True
B. False
C. 0
D. None

Answer: False

Solution: Zero is a falsy numeric value, so bool(0) is False.

Q189. What is bool(False as text)?

A. False
B. True
C. None
D. ValueError

Answer: True

Solution: A non-empty string is truthy regardless of its characters; bool('False') is True.

Q190. What does list(aba) return when aba is a string?

A. ['a', 'b', 'a']
B. ['aba']
C. ('a', 'b', 'a')
D. {'a', 'b'}

Answer: ['a', 'b', 'a']

Solution: list(string) iterates over characters, producing ['a', 'b', 'a'].

Q191. What does tuple(aba) return when aba is a string?

A. ['a', 'b', 'a']
B. ('a', 'b', 'a')
C. {'a', 'b'}
D. aba

Answer: ('a', 'b', 'a')

Solution: tuple(string) iterates over characters and packs them into ('a', 'b', 'a').

Q192. What effect does converting an iterable to a set have?

A. It removes duplicate values
B. It preserves duplicates and order
C. It creates a dictionary
D. It converts every value to a string

Answer: It removes duplicate values

Solution: Set construction keeps only unique values, so duplicate elements are removed.

Q193. What input does dict() accept in the notes example?

A. An iterable of two-value pairs
B. Only a string
C. Only an integer
D. A set of single values

Answer: An iterable of two-value pairs

Solution: dict accepts an iterable of pairs, using each pair's first item as a key and second as its value.

Q194. What does encoding text produce?

A. Bytes
B. A list
C. An integer identity
D. A function

Answer: Bytes

Solution: Encoding translates text characters into a bytes representation using a character encoding.

Q195. What does decoding bytes produce?

A. Text
B. A float
C. A set
D. A code object

Answer: Text

Solution: Decoding interprets bytes with an encoding and produces text (a str).

Q196. What happens in 2 + 3.5?

A. The result is a float
B. The result is an integer
C. The result is a string
D. It always raises TypeError

Answer: The result is a float

Solution: Mixed numeric addition promotes the integer to floating point, so 2 + 3.5 is a float.

Q197. What does ord(A) return?

A. 65
B. 97
C. 48
D. A

Answer: 65

Solution: The Unicode code point for uppercase A is 65, so ord('A') returns 65.

Q198. What does chr(65) return?

A. A
B. a
C. 65
D. None

Answer: A

Solution: chr maps code point 65 back to its character, uppercase A.

Q199. What information can int(3.75) lose?

A. The fractional part
B. The sign always
C. The integer part
D. The type name only

Answer: The fractional part

Solution: Converting 3.75 to int removes the fractional part, producing 3.

Q200. What is the difference between validation and conversion?

A. Validation checks data; conversion changes data
B. They are exactly the same
C. Conversion only checks types
D. Validation always creates bytes

Answer: Validation checks data; conversion changes data

Solution: Validation inspects whether data meets requirements; conversion transforms it into another representation or type.